

Personal Expense Tracker

A modular, standard-library Python command-line application for recording and reviewing personal finances.

Prepared: 30 September 2026 | Repository: github.com/Pankhudi-28/Expense_Tracker

Executive Summary

The Personal Expense Tracker converts the supplied interactive script into a small package with distinct modules for transaction data, JSON persistence, business operations, analytics, and command-line interaction. Users can record income and expenses, maintain categories, search and edit transactions, and review monthly totals. The project runs locally with Python's standard library and stores data in a portable JSON file.

Problem, Audience, and Scope

Manual tracking can make it difficult to see monthly cash flow and category-level spending. This tool is intended for students, individuals, and households who want a lightweight offline ledger and are comfortable running Python commands. It covers one user's local records; it does not connect to banks, synchronize devices, encrypt the data file, or provide financial advice.

Functional Requirements

Module	Delivered capability
Transaction management	Create, list, filter, search, update, and delete income or expense records.
Category management	Use default categories and add custom income or expense categories.
Analytics and reporting	Summarize monthly income, expenses, net balance, category totals, and the highest expense.
Command line	Run actions through <code>python -m expense_tracker</code> and its subcommands.

Architecture

File	Role
<code>models.py</code>	Transaction data model and validation; monetary values are normalized to two decimals.
<code>storage.py</code>	JSON defaults, defensive loading, and atomic file replacement.
<code>service.py</code>	Transaction CRUD and category operations.
<code>analytics.py</code>	Monthly aggregation using decimal arithmetic.
<code>cli.py</code> / <code>__main__.py</code>	Argument parsing, user-facing output, error reporting, and module entry point.
<code>tests/test_tracker.py</code>	Unit tests for CRUD, validation, persistence, and analytics.

Quality and Operation

Requirement	Implementation
Error handling	Validate type, amount, date, and category; raise clear errors for invalid or unreadable databases.
Logging	Use Python logging for storage diagnostics; enable detailed messages with <code>- -verbose</code> .
Performance	Use in-memory filtering and aggregation with linear time in the number of local transactions.
Maintainability	Keep CLI, model, service, persistence, and analytics in focused modules with type hints and docstrings.
Data integrity	Serialize to a temporary file and atomically replace the target database.
Portability	Use Python 3.10+ and standard-library modules only; no package installation is needed.

Run and Test

From the project root, display available commands with `python -m expense_tracker - -help`. Example: `python -m expense_tracker add-expense 250 --category Food --description Lunch`. The default JSON database is `finance_data.json`; change it with `--database` or `EXPENSE_TRACKER_DB`.

The unit test suite is included in the repository. Run it with `python -m unittest discover -s tests -v`. Tests were not executed while preparing this report.

Repository and Deliverables

The application package, updated README, problem statement, and tests are published in the public GitHub repository at github.com/Pankhudi-28/Expense_Tracker. The earlier README was replaced, and the misspelled source script was removed. This report is delivered as a separate PDF and is not stored in the repository.

Limitations and Next Steps

The local JSON database has no encryption or concurrent-writer coordination. Potential extensions include CSV export, budgets, recurring transactions, and optional backup support.

Deliverable status: source files and unit tests are published; test execution remains unverified.